

UML Diagrams

↳ Use Case Diagram

↳ Class Diagrams

↳ Composition v/s Aggregation

Next Class

UML Diagrams

→ Communication

Who do you comm with

→ Tech lead → approach / req

→ Product Manager → req

→ QA / Devops → how system flows.

→ Peers (Fellow SWE) → PR Review, Understanding Flow

→ Client → Req
→ Flow of the System

as software engineer, we have to understand that

1) what should i do

2) and info someone how something is done on which you worked

What should I do

→ how to approach design problems

how something is done

→ how a particular SW system works

→ what are pieces of a SW system etc.

Ways to Comm

Words

- ↳ Email
- ↳ Meeting
- ↳ Chat
- ↳ Doc

Cons

- ↳ Miscomm
- ↳ Diff to be visualized
- ↳ Issues when you are explaining a complex concept

V/S

Visuals

/ Pictures / Flowchart / Diagrams

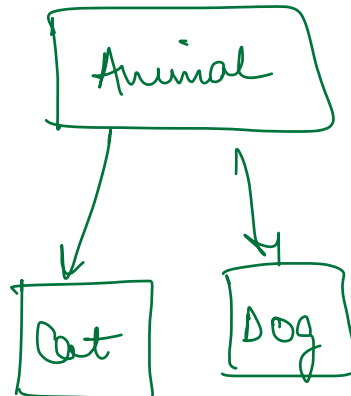
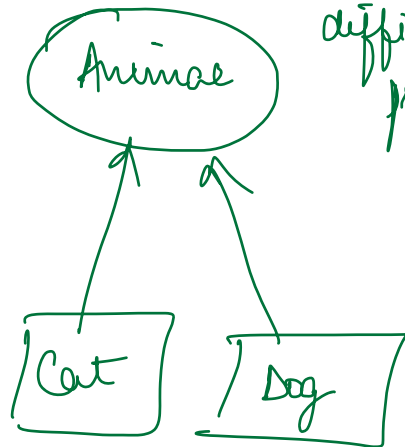
→ no need to visualize

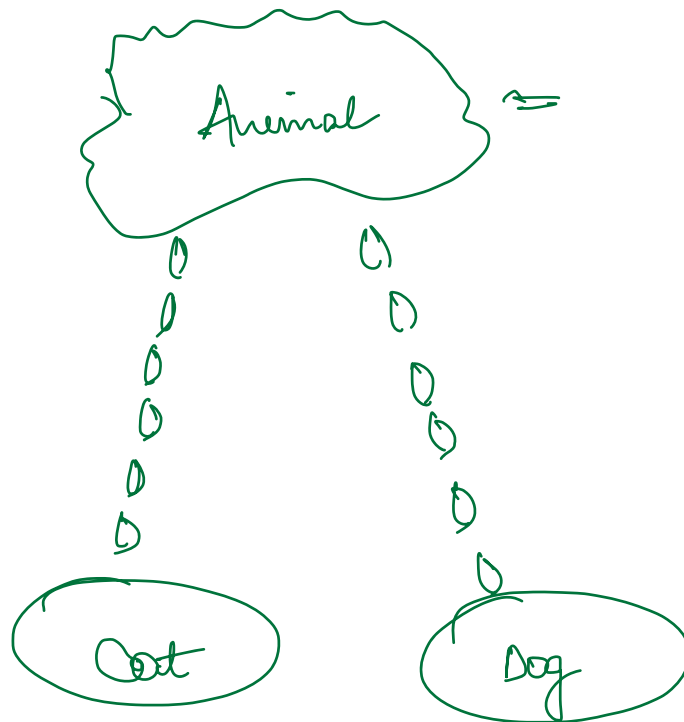
Anything

→ lesser scope of ambiguity

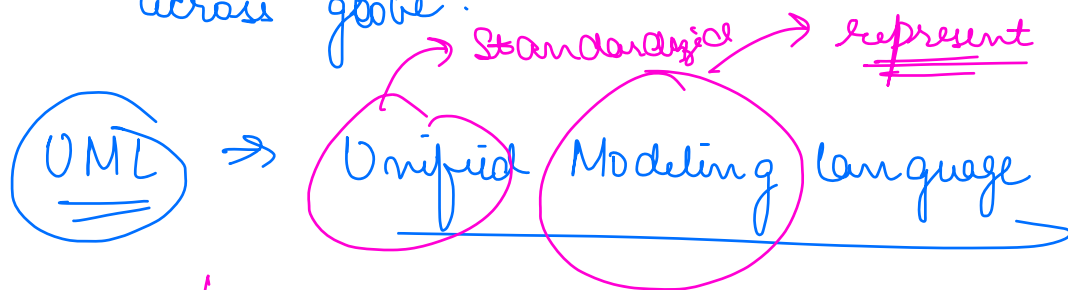
But → No uniformity / No standardization

→ understanding difficult / problematic





⇒ There is a need for standardization in diagrams to ensure proper understanding across globe.



↳ standardization on how to rep
diff SWE concepts as diagrams

- class
- interface
- abs class

→ because in UWE there are further many concepts, there are diff types of UML diagrams.

Diff Types of UML Diag

Structural UML Diagrams

→ structure
How a codebase is structured

→ structure of codebase
→ what are diff classes
→ how are classes talking to each other.

→ rel^m of code entities -
~~only UML diag asked in interview~~

- ① Class Diagram
- ② Package Diagram
- ③ Object Diagram
- ④ Component Diagram

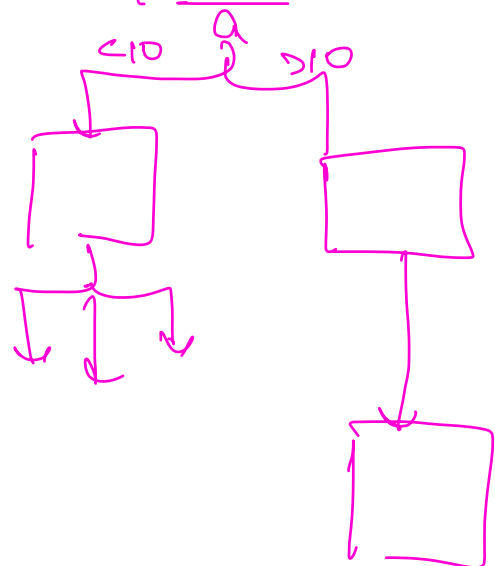
H/W

Behavioural UML Diagrams

→ behaviours
How a flow system works.

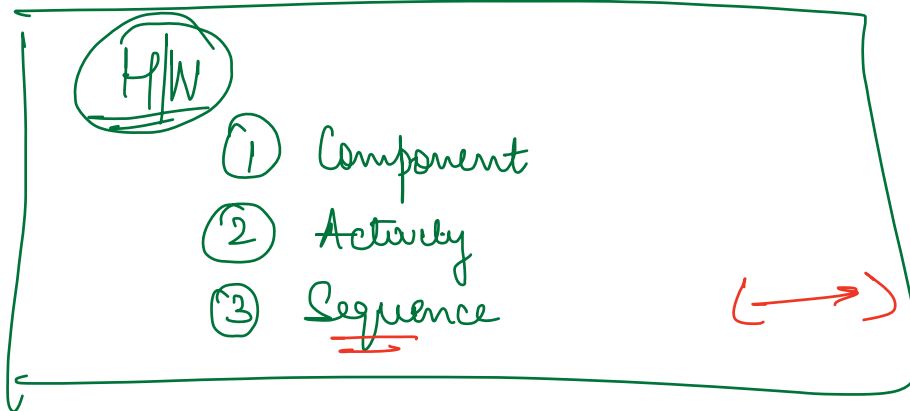
→ how a particular method works.

↳ Flow Chart

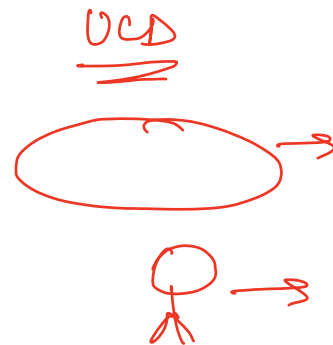
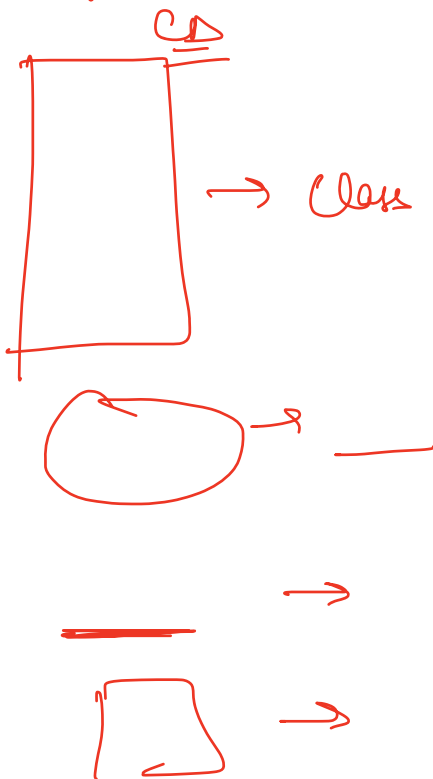


→ what are diff features supp by a flow system

- H/W ~~1~~ ① Activity Diagram
 Today's ~~2~~ ② Use Case Diag
 H/W ~~3~~ ③ Sequence Diag



every UML diag is a set of figures rep diff things



USE CASE DIAGRAM

- Represents diff features/functionalities supported by a SW system.
- Any SW system consists of a large # of features

Figures: 15 comp figures & 3 figures + 2 adv things

①

System Boundary

- there are some f^n that are used by a SW system that are not impl within it (but impl in some external system)
- create a boundary b/w what is supp by your system and what is not ~~of~~ supported by your system
- rep using a 
- every f^n that is supported by my system will go inside boundary.



II

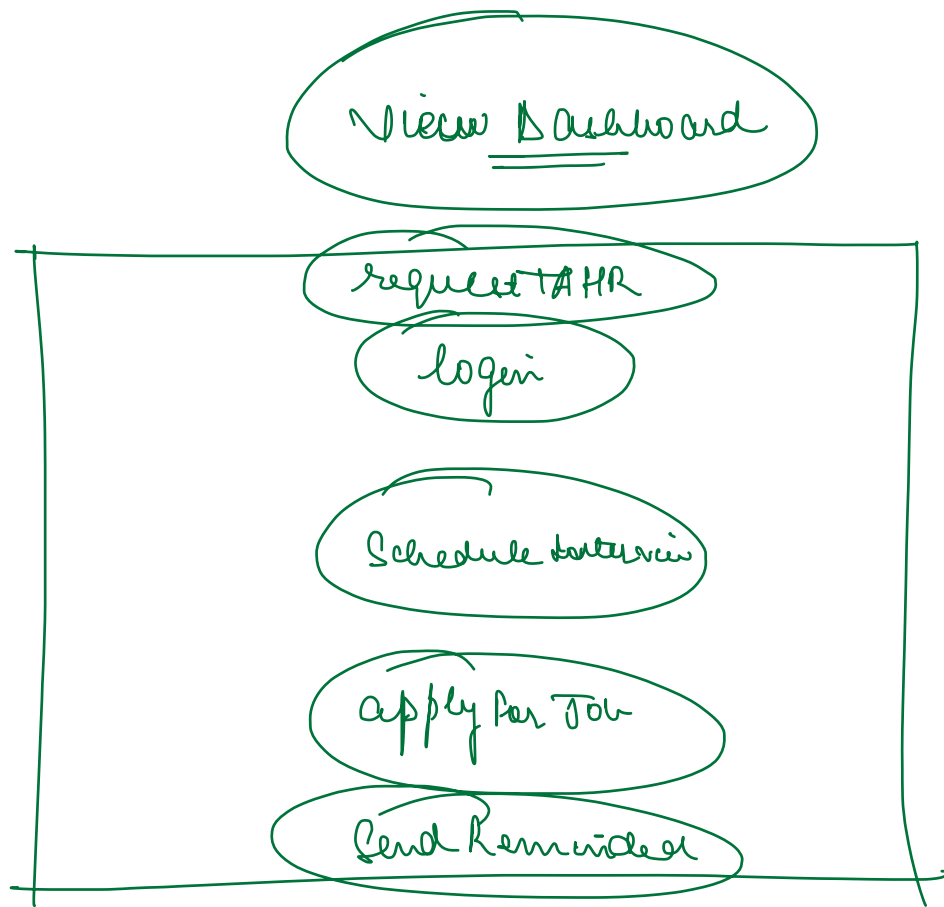
Use Case

→ ⁿ feature / action / functionality
that is supported by my s/w
system

→ keep using an oval inside which
you put the name of the feature

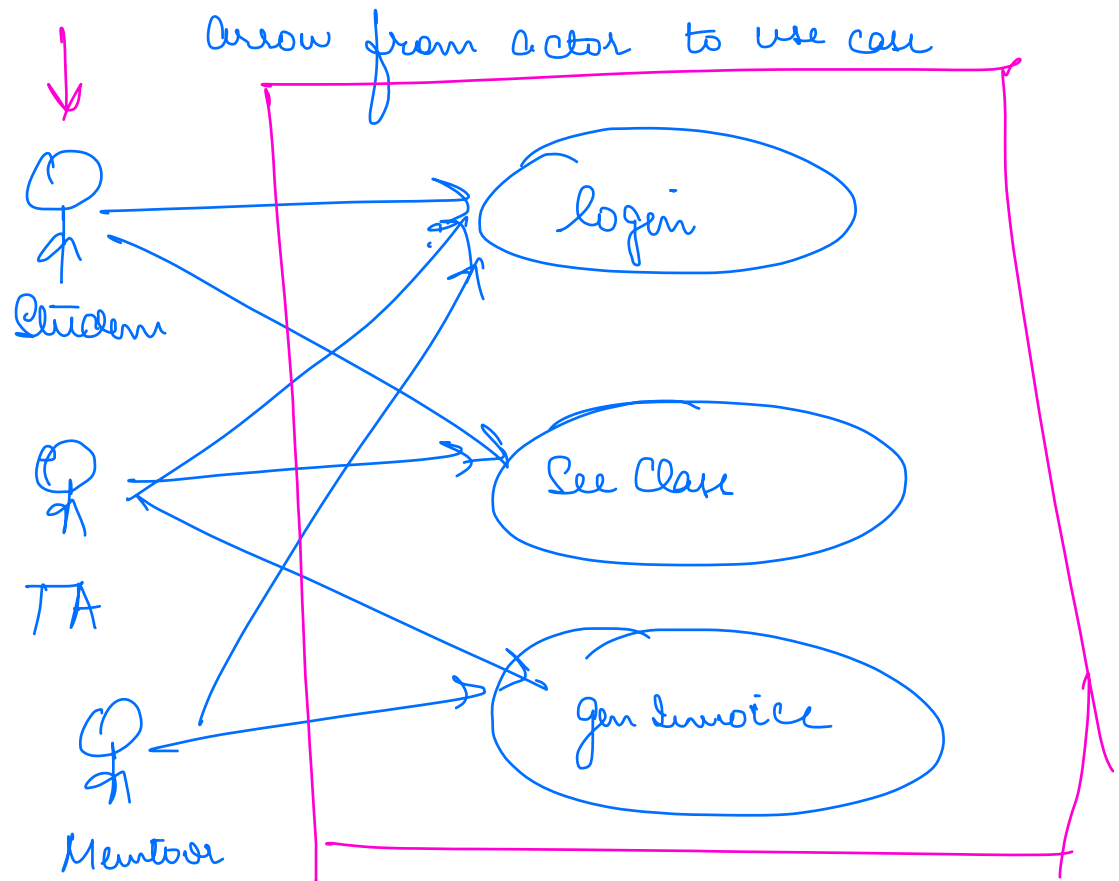
login

→ must always be a verb



III Actor

- Someone who can use diff use cases in a s/w system.
- Actor must always be Noun
- Rep using a 
- eg Instructor, TA, Mentor, Student
- a relⁿ from actor to use case
it can use it rep via an



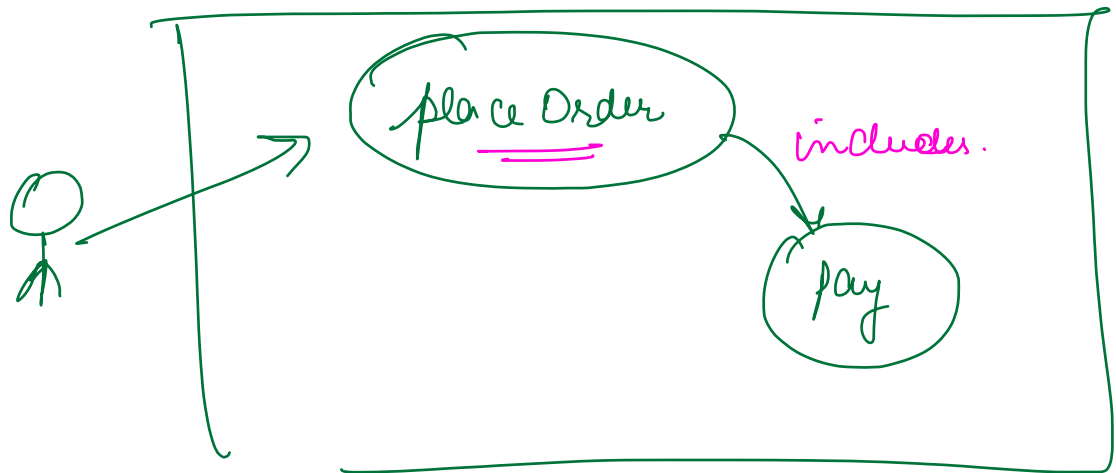
→ always outside the sys boundary.

④ Includes

→ are all use cases implemented in a S/W
Sep from each other.

→ NO

→ One use case may call another use case.



⇒ Checkout() {

← Think of every use case as a method

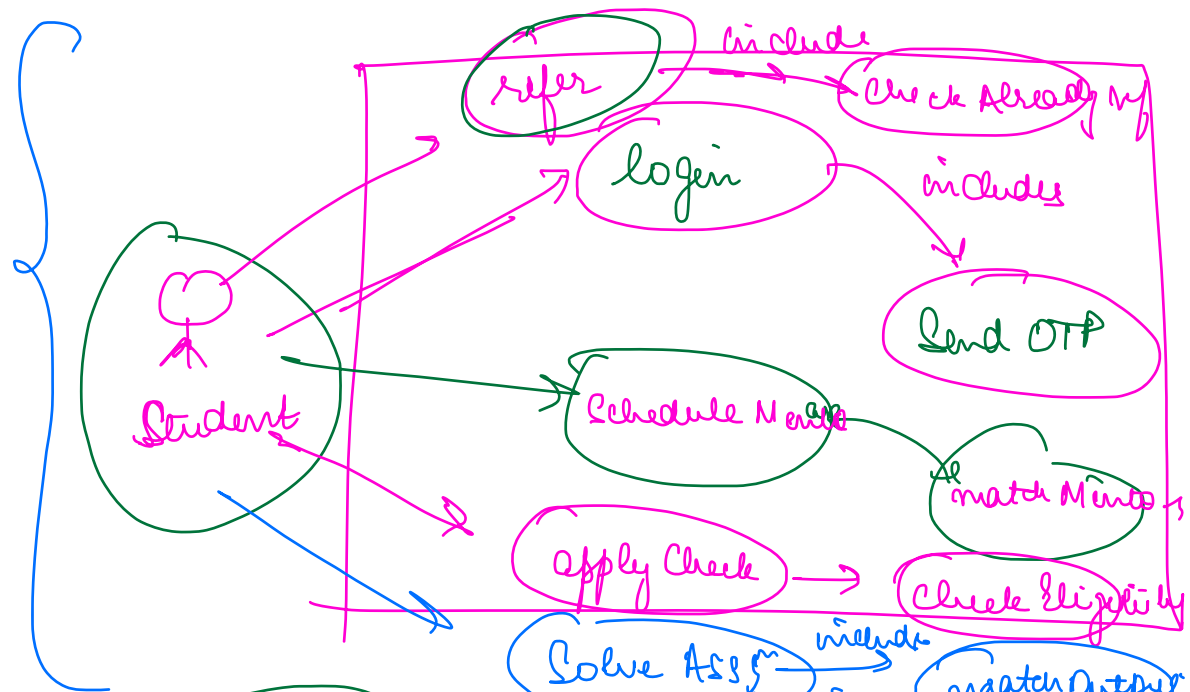
Verify Address()
Check Inventory()

}

When one method / use case ^(A) calls other use case ^(B) (depends on other use case)
Other use case has to be completed to complete it

⇒ A includes B

refer() {
 check Already
 Refer B
}



```

join class C {
    <eat>
}

```

→ include is not an
order relⁿ
 → it is dependency

Not
includes

```

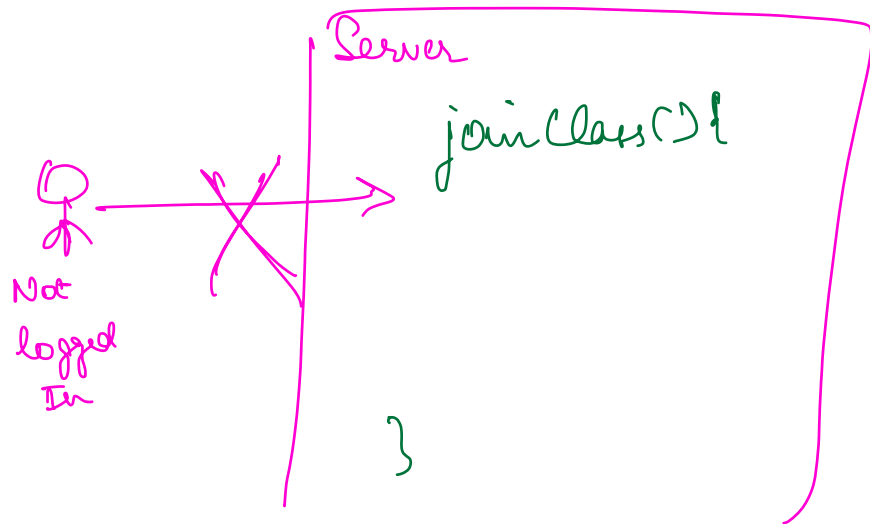
View Dashboard() {
    < login()
}

```

```

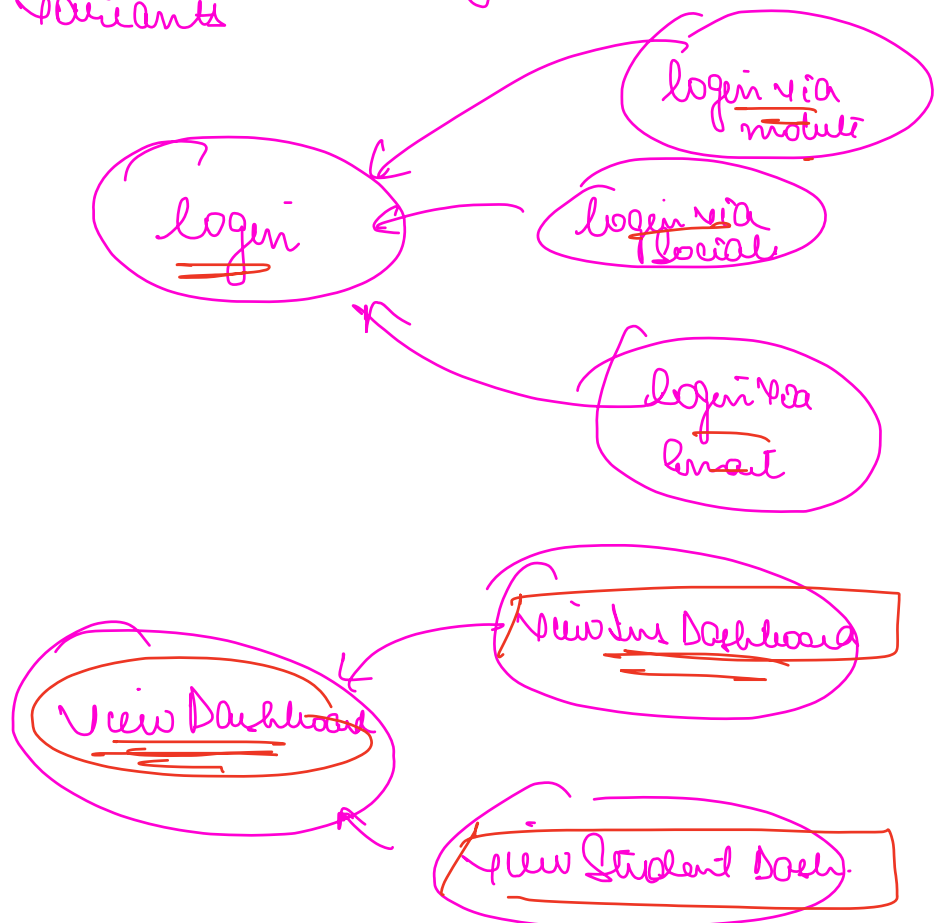
a() {
    b();
    c();
}
a -- includes --> b
a -- includes --> c

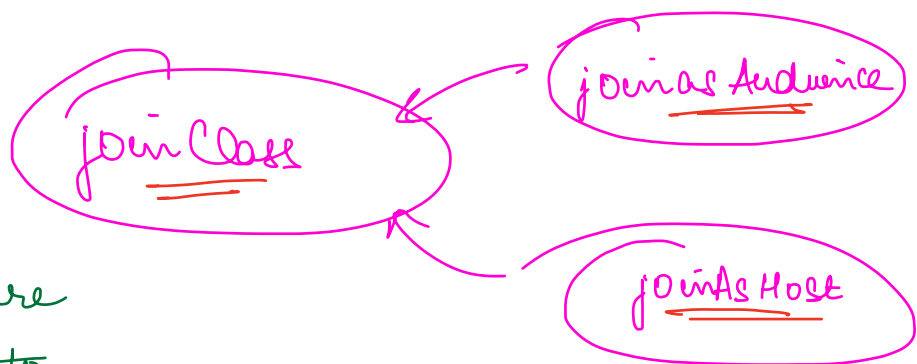
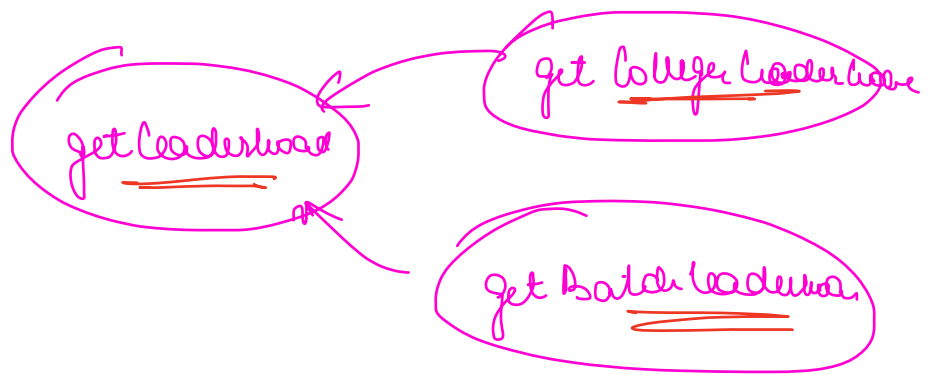
```



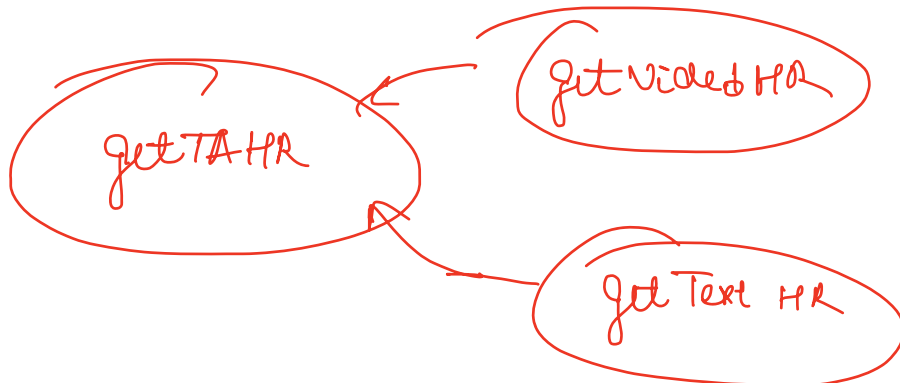
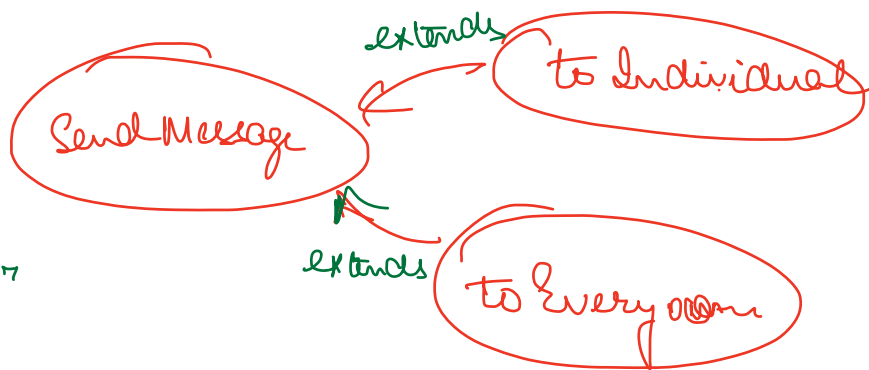
⑤ Extends

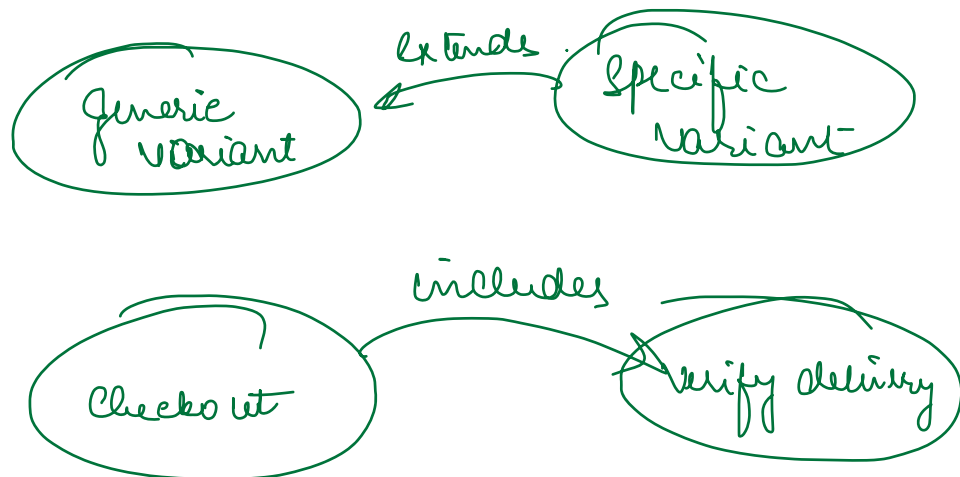
→ One use case may have multiple variants





When there are multiple of n to perform a particular f^n , think of "extends"



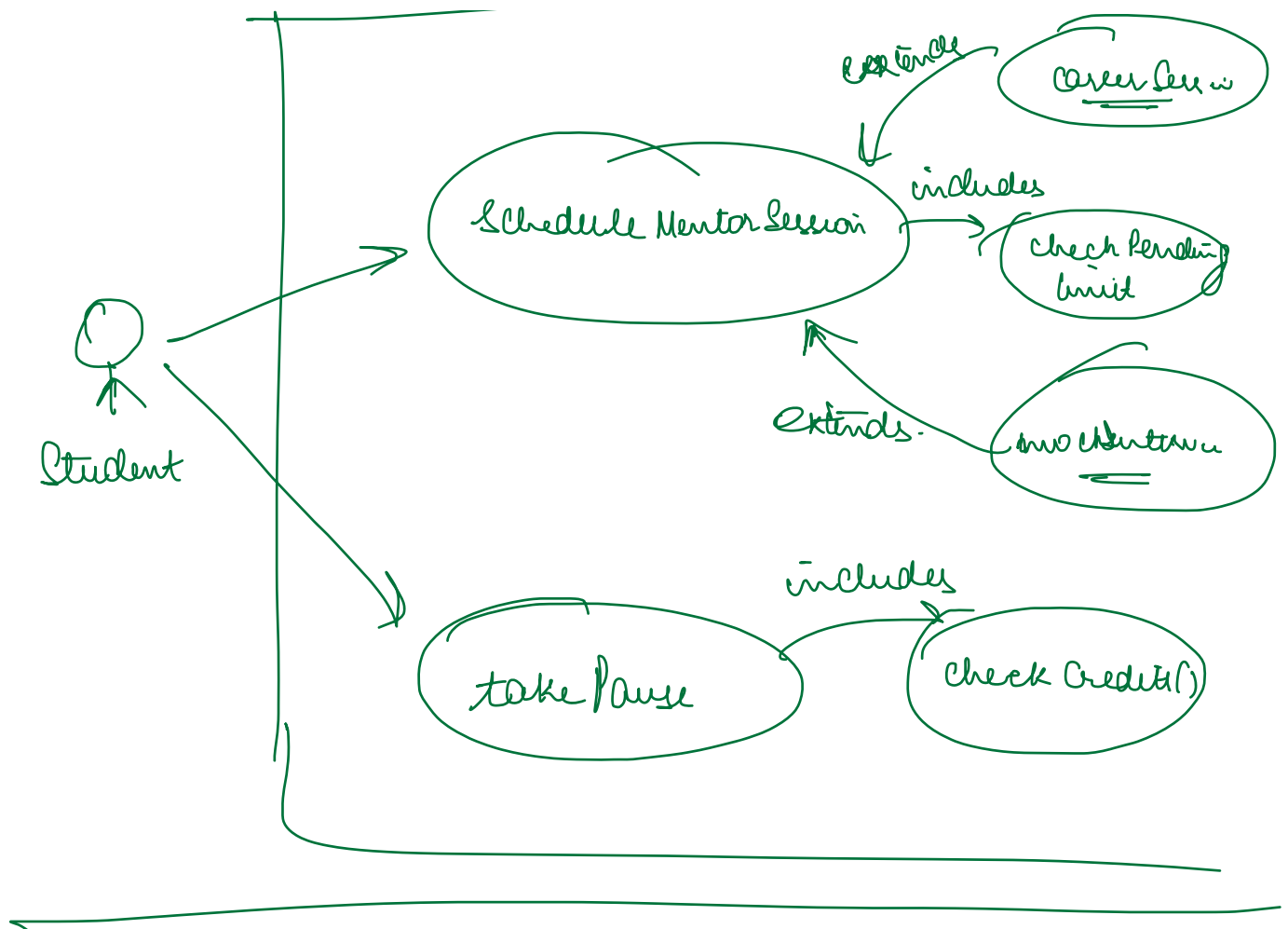


Q^m

Draw UC Diagram of Scaler

- a-) atleast 5 use cases
- b-) — 2 actors.
- c-) — 1 use case must have specific variants
- d-) — 1 use case must include another use case.
- e-) None of these should be what we have already found in the class.

:



CLASS DIAGRAM

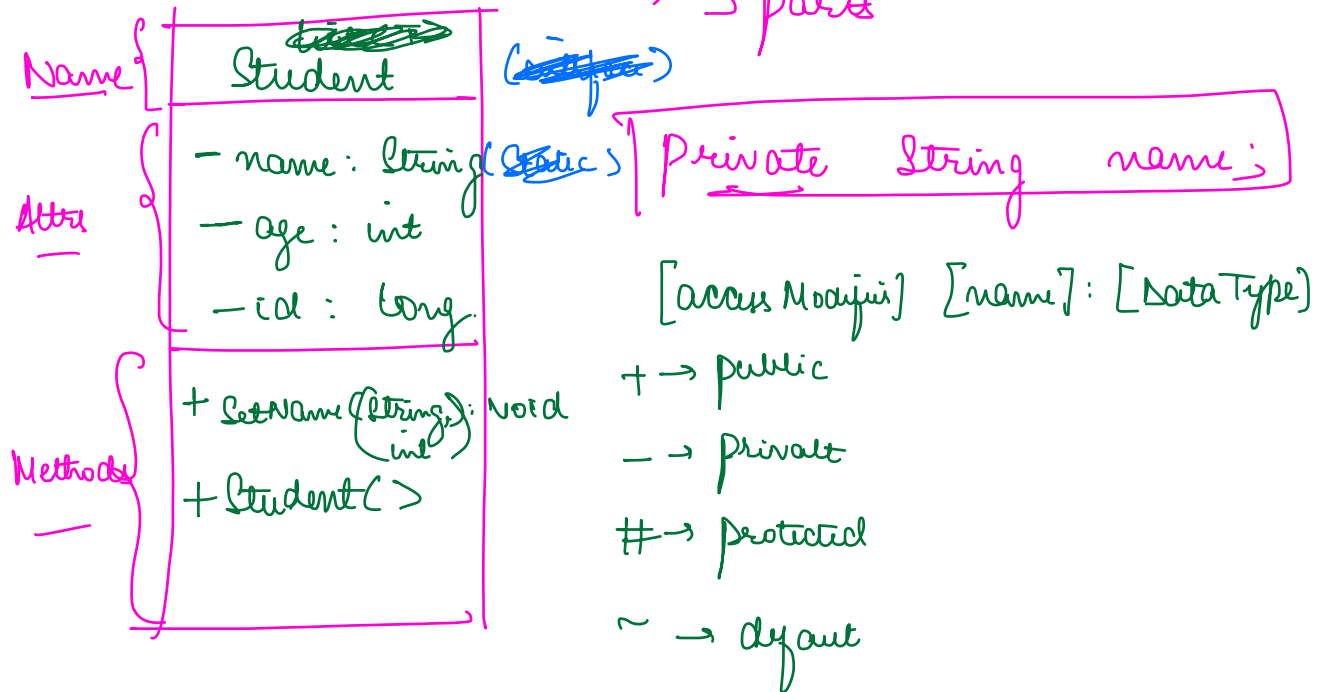
→ keep diff types of Entities that are there in your S/W System

- class
- interface
- abs class
- enum

→ And relⁿ b/w those entities

I Class → rectangle

→ 3 parts



```
public void setName(  
    String name);
```

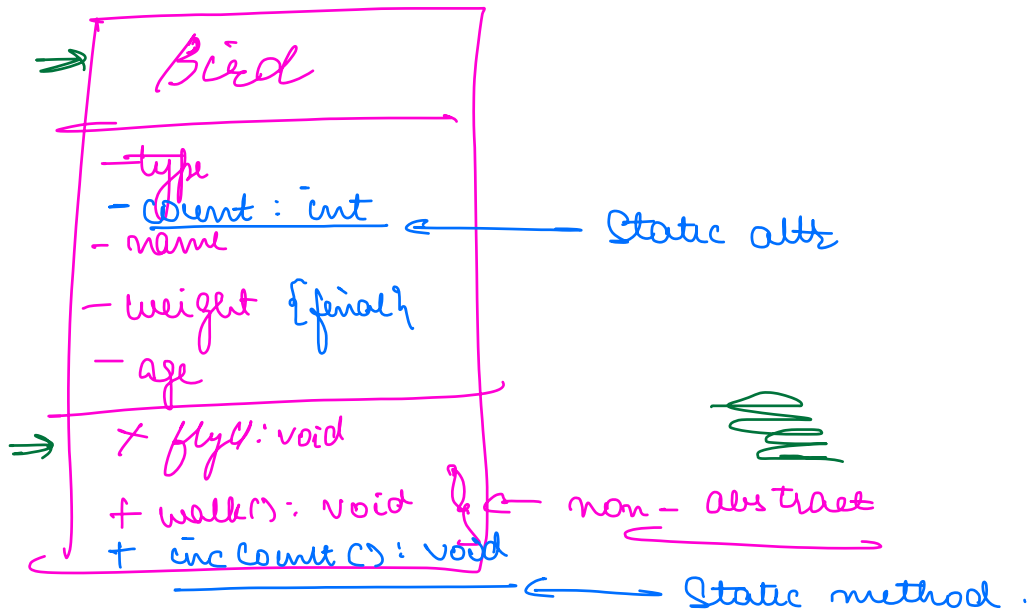
[access modifier] [name] ([data types of parameters]): [return data type]

II Abstract Classes

→ methods

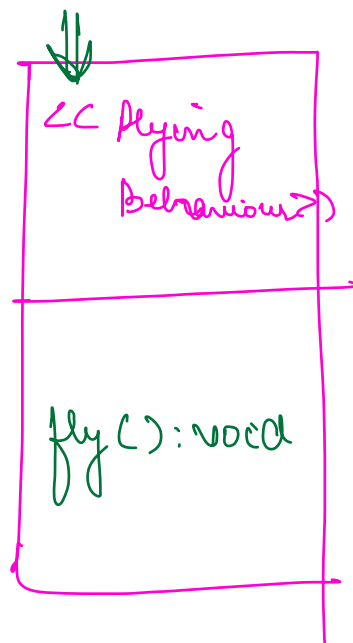
⇒ exactly like a class BUT

⇒ everything that is abstract should be static



III Interfaces

<<Serializable>>



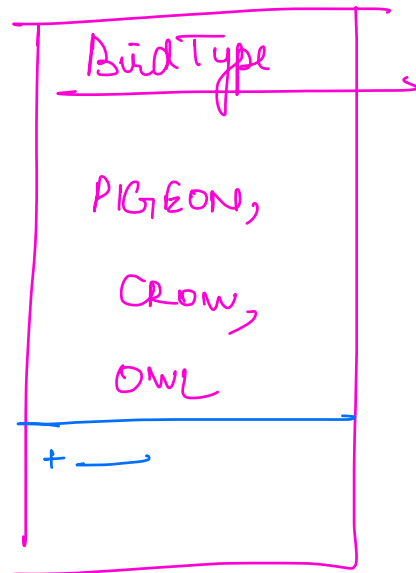
→ name is put in angle brackets

→ methods are rep as they are in a class, just no access modifier

IV Enums

→ rectangle with 2 parts

→ each value of enum is rep by a constant



~~XXX~~

How will you rep something that is static
→ underline on the method or attribute